AI-SRS Compliance Analyzer

System Architecture, Component Interactions & Developer Onboarding Guide

---

Target Audience: *Incoming Software Engineers, AI/ML Interns, and Project Stakeholders*
Primary Technologies: *Python 3.13, PyMuPDF, Sentence-Transformers (MiniLM), DeBERTa-v3 NLI, Rank-BM25*
Repository Status: *Complete End-to-End V2 Pipeline with Declarative Benchmarks (151 passing tests)*

1. What Does This Project Do? (Plain English)

When engineering teams build complex software or scientific systems, they write a **System Requirements Specification (SRS)** or a technical report. Companies and regulatory bodies have strict checklists and reference standards (e.g., IEEE, ISO, safety guidelines, lab rubrics) that these documents must comply with.

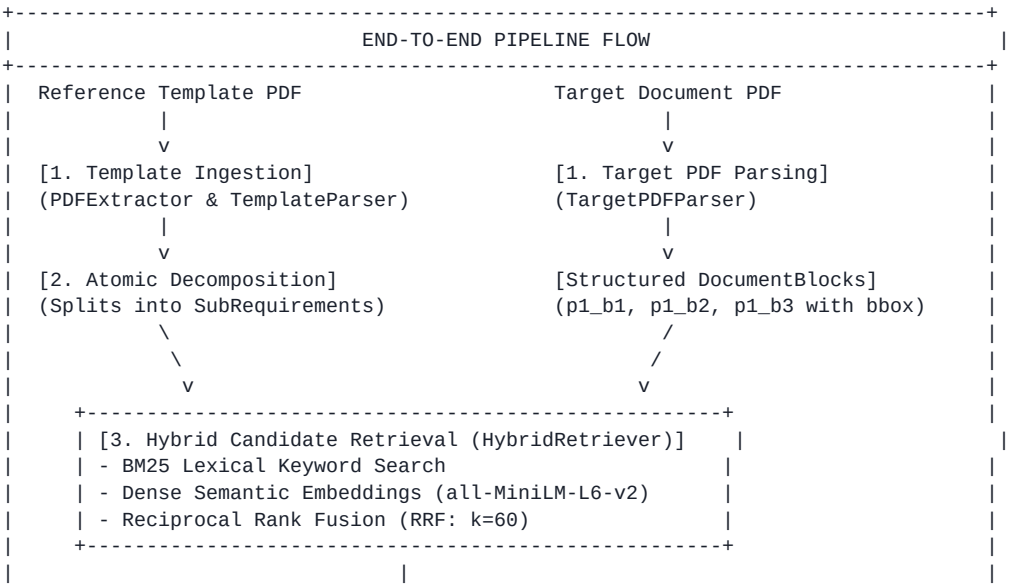
Traditionally, auditing an SRS is a painful manual process: an engineer or auditor reads a 20-page document line-by-line with a checklist, searches for matching sentences, verifies numbers, checks whether required features were omitted, and manually writes an audit report. It is slow, subjective, and error-prone.

The AI-SRS Compliance Analyzer automates this entire audit process:

- **Input 1 (Reference Template PDF):** The standard, checklist, or rubric containing required sections and rules.
- **Input 2 (Target Document PDF):** The candidate SRS, lab report, or technical specification being audited.
- **Automated Output:** An executive compliance report with an overall verdict (COMPLIANT, CONDITIONALLY_COMPLIANT, NON_COMPLIANT), mandatory coverage percentage, and line-by-line evidence proof with exact page numbers and quotes.

2. How Does It Work? The 5-Stage Pipeline

The system executes a deterministic 5-stage pipeline orchestrated by src/pipeline.py:



```
|                              v                                                  |
|     +-------------------------------------------------+                       |
|     | [4. NLI Evidence Verification (EvidenceVerifier)]   |                    |
|     | - Cross-Encoder Transformer (nli-deberta-v3-small)  |                    |
|     | - Premise = Candidate Block, Hypothesis = Claim    |                    |
|     | - Refinements: Boundary Clauses, Meta-Discourse     |                    |
|     | - Classifies: FULFILLED, CONTRADICTED, OMISSION, ...|                    |
|     +-------------------------------------------------+                       |
|                              |                                                 |
|                              v                                                 |
|     +-------------------------------------------------+                       |
|     | [5. Compliance Aggregation (ComplianceAggregator)]  |                    |
|     | - Rolls up sub-results to parent sections           |                    |
|     | - Computes mandatory & optional coverage %          |                    |
|     | - Computes document verdict (COMPLIANT / NON)       |                    |
|     +-------------------------------------------------+                       |
|                              |                                                 |
|                              v                                                 |
|     [Terminal ANSI Summary] [compliance_report.md] [compliance_report.json]     |
+--------------------------------------------------------------------------------+
```

3. Deep Dive into the Core Components (With Code Snippets)

Stage 1: Document Parsing & Layout Chunking (src/parsers/pdf_parser.py)

Instead of reading raw, messy text strings, TargetPDFParser parses the PDF layout. It groups contiguous spans of text into cohesive DocumentBlock objects and assigns each a deterministic identifier like p1_b3 (Page 1, Block 3). It classifies each block as a HEADING, PARAGRAPH, LIST_ITEM, TABLE_ROW, or DISCLAIMER.

```
@dataclass
class DocumentBlock:
    block_id: str                          # e.g., 'p1_b3'
    text: str                              # Clean text content
    block_type: BlockType                  # PARAGRAPH, HEADING, DISCLAIMER
    page_number: int                       # 1-based source page
    section_heading: Optional[str] = None  # Parent section title
    is_explicit_omission: bool = False     # True if text states 'not included'
```

Stage 2: Requirement Decomposition (src/parsers/requirement_decomposer.py)

A single human requirement in a checklist often contains multiple testable claims (e.g., 'State the purpose and describe what it is intended to measure'). The RequirementDecomposer breaks parent points into atomic SubRequirement claims, each with its own expectation type (FACTUAL_STATEMENT, NUMERIC_TARGET, SAFETY_CONSTRAINT, or INTERFACE_SPECIFICATION).

```
@dataclass
class SubRequirement:
    sub_id: str                          # e.g., 'B1.1', 'B1.2'
    parent_point_id: str                 # e.g., 'B1'
    claim_text: str                      # Atomic verifiable hypothesis
    expected_evidence_type: EvidenceType   # FACTUAL_STATEMENT, NUMERIC_TARGET...
    is_mandatory: bool = True            # Mandatory vs Optional flag
    decomposition_rationale: str = ""
```

Stage 3: Hybrid Retrieval with Reciprocal Rank Fusion (src/retrieval/)

Why Hybrid? Keyword search alone fails when documents use synonyms ('cooling rate' vs 'temperature decrease'). Dense embedding search alone fails when matching exact acronyms, numbers, or section codes ('ISO 26262', '70 °C'). The HybridRetriever runs both and mathematically fuses their ranks using **Reciprocal Rank Fusion (RRF)**:

```
# Reciprocal Rank Fusion formula with smoothing factor k=60:
rrf_score = 1.0 / (60 + rank_bm25) + 1.0 / (60 + rank_dense)
```

```
# Retrieves Top-K fused candidates for each atomic sub-requirement claim
candidates = retriever.retrieve_candidates_for_subrequirement(sr, top_k=5)
```

Stage 4: AI Evidence Verification via NLI (src/verification/evidence_verifier.py)

The system does not rely on naive keyword matching. It feeds the retrieved candidate passage and the requirement claim into a cross-encoder Natural Language Inference transformer (nli-deberta-v3-small). The model computes probabilities for **Entailment** (proof found), **Contradiction** (conflicting claim), and **Neutral** (unrelated text).

- **FULFILLED:** Entailment probability exceeds threshold (e.g. >0.70).
- **CONTRADICTED:** Evidence directly opposes the requirement.
- **EXPLICIT_OMISSION:** Document contains an explicit disclaimer statement (e.g. 'Safety precautions are not covered').
- **MISSING:** No relevant or entailing evidence found in candidate blocks.
- **NEEDS_REVIEW:** Conflicting signals or ambiguous evidence flagged for human inspection.

Stage 5: Hierarchical Compliance Aggregation (src/verification/compliance_aggregator.py)

The aggregator rolls up granular atomic decisions into parent sections (FULLY_COMPLIANT, PARTIALLY_COMPLIANT, NON_COMPLIANT, NEEDS_REVIEW) and calculates exact mathematical coverage metrics:

```
# Mandatory Coverage formula (rounded to 1 decimal place):
mandatory_coverage_pct = round((mandatory_fulfilled / mandatory_total) * 100.0, 1)

# Document Verdict Rules:
if any_mandatory_has_needs_review:
    verdict = DocumentComplianceVerdict.CONDITIONALLY_COMPLIANT
elif mandatory_coverage_pct == 100.0 and no_mandatory_contradictions:
    verdict = DocumentComplianceVerdict.COMPLIANT
else:
    verdict = DocumentComplianceVerdict.NON_COMPLIANT
```

4. The Declarative Benchmark Suite (Phase 2.6)

To guarantee that future AI model updates or code refactorings never degrade compliance accuracy, the project includes an automated benchmark evaluation harness. Each test pair has a declarative JSON manifest (e.g., data/benchmarks/pair_b_srs.json) defining ground-truth evidence blocks and expected statuses.

The **Three-Layer Validator** (src/benchmarks/manifest_validator.py) validates all manifests:

- **Layer A (JSON Schema):** Enforces JSON Schema Draft 2020-12 structural constraints and data types.
- **Layer B (Cross-Field Logic):** Enforces business invariants (mandatory_total >= 1, ID hierarchy, status conservation).
- **Layer C (Document Integrity):** Parses the physical PDFs, checks 64-char SHA-256 digests, and validates that every quoted evidence excerpt actually exists word-for-word in the target PDF.

5. Step-by-Step Developer Guide: How to Run & Test

Here are the exact commands the incoming intern needs to get started, run tests, and execute audits:

Step 1: Environment Setup

```
# 1. Clone repository
git clone https://github.com/Shubh-4563/SRS---Compliance---analyzer.git
cd SRS---Compliance---analyzer

# 2. Create virtual environment (Python 3.10+ recommended, tested on 3.13)
python3 -m venv .venv
source .venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt
```

Step 2: Run the Full Test Suite

```
# Runs all 151 unit and regression tests across all subsystems:
python3 -m unittest discover -s tests -p "test_*.py" -v

# Expected Output: Ran 151 tests in ~25-30s ... OK
```

Step 3: Run the Live Pipeline on Pre-Built Test Pairs

```
# Evaluate Pair B (Equipment Monitoring Dashboard vs Technical Template):
python3 main.py \
  --template data/test_packs/B_reference_technical_template.pdf \
  --target data/test_packs/B_target_technical_document.pdf \
  --format all

# Output generated:
# - Terminal ANSI summary
# - compliance_report.md (formatted Markdown audit document)
# - compliance_report.json (machine-readable data with exact quotes)
```

Step 4: Run the Automated Benchmark Accuracy Runner

```
# Evaluates accuracy across all declarative benchmark pairs:
python3 scripts/evaluate_manifest_benchmark.py --all

# Evaluates Hit@1..5, MRR, Accuracy (100%), and zero discrepancies
```

Step 5: Testing a New Document from Ma'am or Customer

```
# Place the reference checklist PDF and candidate document PDF in data/:
python3 main.py \
  --template data/my_checklist.pdf \
  --target data/my_candidate_srs.pdf \
  --output-dir reports/my_audit/ \
  --format all
```

6. Future Development Roadmap for the Next Intern

Here are high-priority directions the next intern can take forward:

1. **OCR Integration (Tesseract / EasyOCR):** Add support for scanned paper PDFs or photos of requirements documents (currently only digital PDFs with selectable text are supported).
2. **Large Document Scaling (Pair C & D):** Benchmark against multi-page industrial SRS documents (50+ pages) to stress-test hybrid retrieval candidate ranking.
3. **Interactive Web UI (Streamlit / FastHTML):** Build a drag-and-drop web dashboard where users can upload two PDFs and view interactive compliance findings.
4. **Model Optimization:** Test quantized or ONNX models (e.g. MiniLM-L6 ONNX) to decrease CPU inference latency for faster local audits.

Document compiled for internship onboarding and technical handover. Source code and git history are synchronized at origin/main.